Day-07 ~~Mathmatical Assignment~~

Theory Assignment

Problem 1: The matrix calculus layout Divide

Task A: Define the numerator layout and the Denominator layout. Specially, if you derivative an $m \times 1$ vector y w.r.to an $n \times 1$ vector x, what are the resulting Dimensions under each layout.

Task B: In modern DL framework (like pytorch), we use the Denominator layout when taking the derivative of the final scalar loss L w.r.to a weight matrix W. Theoretically, why is this convention strictly preferred over the Numerator layout for updating weights?

Solution: Task A:

In numerator, differentiating is $m \times n$ Jacobian matrix.

CamScanner

Task B:
Denominator layout ensure that the gradient of the $\nabla_W L$ has the exact same dimensions as the original matrix ~~directly~~ W. This allows the optimizer to simply subtract the gradient matrix directly from the weight matrix without manual transpositions.

Problem 2: The Anatomy of a Computational Graph.
Nueral networks are executed as DAG rather than traditional monolithic mathmatical equations.

Task A: Define the specific purpose of the Forward pass. What crucial pieces of information must be cached during the forward pass at every node and why?

Task B: During the Backward pass, every node performs the operation. Downstream = up stream X Local. Define what the upstream Gradient and Local gradient represent conceptually.

CamScanner

Task C: How does an Addition (+) node behave during the backward pass compared to a multiplication (×) node?

Solution:

Task A: The forward pass computes the predictions of the model. It must cache all intermediate activation value at every node because those specific numbers are required to calculate the local gradients during the backward pass.

Task B: Upstream Gradient is the error passed Backward from the layer ahead. Local Gradient is the analytical derivative of the node's specific mathmatical operation.

Task C: An Addition node act as a gradient router, passing the exact upstream gradient

equally to all inputs. A multiplication node act as a gradient switcher, scaling the gradient for one input by the forward-pass value of the other input.

Problem 3: The Necessity of Analytical Gradients.

Task A: If you use the numerical gradient approximation method above, exactly how many forward passes through the network must you compute to find the gradient for all 1,000,000 weights?

Task B: If you see the Back prop algo, how many forward passes and back pass are required to find the exact same gradients?

Task C: Based on your answer to Task A & Task B, summarize why Back prop is the only computationally

viable method for training modern AI.

Solution:

Task A: 1,000,001 forward passes, you must compute the base loss once, and then tweak each of 1,000,000 weights individually by h and re run the entire network to measure the change.

Task B: exactly 1 forward pass & 1 Back pass.

Task C: Numerical gradient have a O(N) time complexity Back prop has O(1).

Training a modern AI without Back prop would literally take thousands of years.

—— X ——